

1 From Mobility Systems to Learning-Based Control

Learning objectives

After studying this chapter, the reader should be able to:

1. explain how mobility differs from the automobile and how connected, automated, shared, and electrified technologies change mobility;
2. locate representative control problems at the mobility-system, vehicle, and component levels;
3. formulate a mobility problem as an optimal control problem, explain why its direct solution may be difficult, and describe the organizing role of Bellman optimality and dynamic programming (DP);
4. identify where approximation or learning enters value information and Bellman computations, a policy, a model, a state estimator, or a combination of these objects; and
5. distinguish offline training, online planning, and online learning, while translating carefully between control and reinforcement learning (RL) terminology.

Chapter roadmap

This chapter moves from the mobility context to the control and learning questions studied in the rest of the book. It first explains connected, automated, shared, and electrified (CASE) mobility and the connected, automated, and electrified vehicle (CAEV). It then introduces the three control levels through representative optimal control problems (OCPs), and uses those examples to explain why approximation is needed, where approximation or learning enters, and when the resulting objects are prepared, used, or revised. Figure 1.1 summarizes this path.

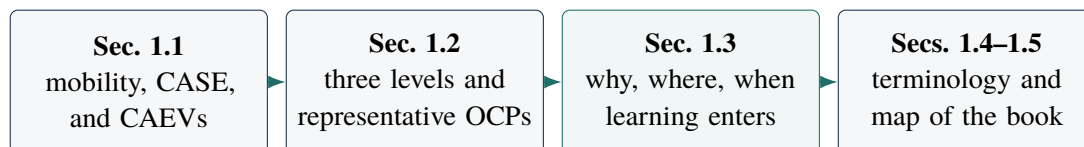


Figure 1.1: Roadmap of Chapter 1, from the mobility context and representative OCPs to learning-based control and the terminology used throughout the book.

1.1 Mobility, CASE, and the CAEV Control Landscape

Mobility is broader than the automobile.

In everyday automotive discussion, *mobility* is sometimes used almost as a synonym for the automobile or for a new vehicle technology. The concepts are not equivalent. An automobile is one means of transport. Mobility concerns the movement that people and goods experience, together with the means, infrastructure, and services that make that movement possible. Different stakeholders emphasize different parts of this broad system: a traveler may focus on access, time, reliability, and comfort; an operator on service and fleet utilization; an infrastructure manager on flow and safety; and a vehicle engineer on the machine that realizes motion.

For this book, a useful formal starting point is the definition adopted in the Republic of Korea's mobility legislation:

Mobility is the act, function, or process of moving or transporting people or goods from one place to another, made possible through the associated means, infrastructure, and services.

The wording is translated and slightly condensed from the official Korean text (Republic of Korea 2023, Art. 2). It makes the scope explicit. This textbook will specifically focus on vehicle-centered control problems, but it does so inside this wider mobility system. Figure 1.2 illustrates this broader boundary through people and goods, multiple transport modes, infrastructure, energy, and digital services.



Figure 1.2: Mobility encompasses the movement of people and goods through multiple modes, supported by infrastructure, energy, information, and services.

Mobility transition through CASE

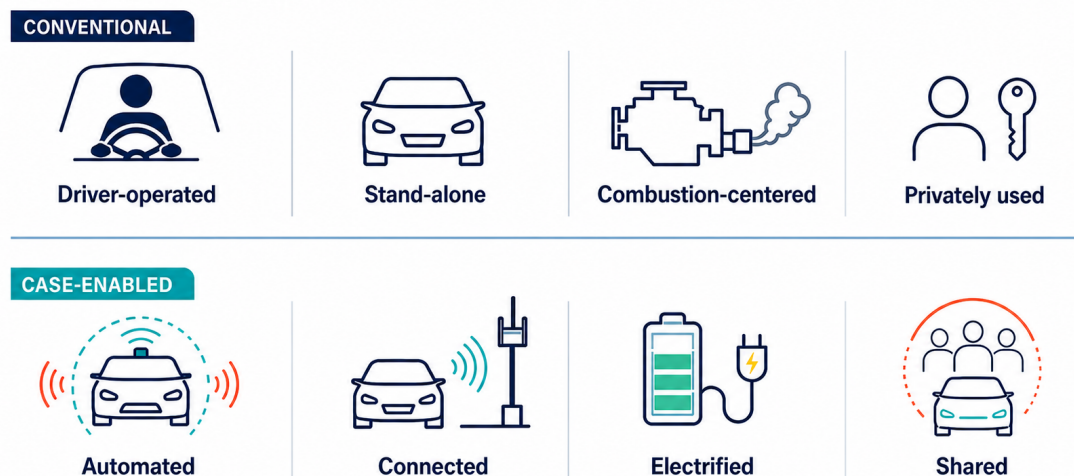


Figure 1.3: Transition from a conventional automobile toward mobility enabled by the four CASE directions.

CASE technologies enable a transition in mobility.

Connected, automated, shared, and electrified technologies are commonly grouped as **CASE**. CASE is not itself a complete definition of mobility. It is a compact description of four mutually reinforcing technology and operation trends that change how mobility can be delivered. A conventional automobile can be understood as predominantly stand-alone, driver-operated, privately used, and combustion-powered. The four CASE directions broaden that starting point, as illustrated in Figure 1.3. The diagram synthesizes the four-axis transition concept introduced in the Siemens mobility material (Siemens Digital Industries Software 2020).

The four directions and the capabilities contributed by each are summarized in Table 1.1.

These capabilities reinforce one another. Connectivity expands the information available to the system, automation transforms available information into autonomous and coordinated decisions and actions, sharing broadens the scope of operation and resource allocation, and electrification enables efficient energy use together with fast, precise, and distributed actuation. None of the four is merely an add-on feature. Their interaction is what moves the engineering question from a stand-alone automobile toward a mobility system.

This book focuses on CAEVs.

The main physical subject of the book is the **connected, automated, and electrified vehicle (CAEV)**. Shared operation remains important when demand, fleet assignment, or common resources enter a mobility-system problem, but it does not define an additional physical control level in this book. Put differently, a CAEV is a vehicle platform through which much of CASE can be implemented; shared mobility is one important way of operating such platforms.

Table 1.1: Four CASE directions and the capability each adds.

Direction	Central capability	What changes from a conventional automobile?
Connected	Exchange information with vehicles, infrastructure, maps, cloud services, and mobility operators.	A decision need not depend only on local onboard sensing; it can anticipate signals, road grade, traffic, hazards, and other agents' intentions.
Automated	Perform perception, planning, decision, and actuation through an automated feedback system.	The perception–decision–control loop shifts from the human to the vehicle, enabling faster, more precise, and consistent driving actions.
Shared	Allocate vehicles, rides, space, charging, or other resources among multiple users and missions.	The unit of operation can shift from one owner and one trip to fleets, demand-responsive service, and utilization across trips.
Electrified	Convert, store, recover, and control energy through batteries, fuel cells, electric machines, power electronics, and electric auxiliaries.	Higher energy efficiency, regenerative energy recovery, and reduced local operating emissions become possible, together with fast, precise, and distributed actuation.

The same CAEV participates in several coupled control layers. Figure 1.4 provides the visual anchor used throughout this chapter. The mobility-system level coordinates information and decisions across vehicles, infrastructure, and services; the vehicle level coordinates actuators to shape the motion of the complete vehicle; and the component level realizes the requested force, torque, or energy conversion in individual devices. The levels are classified primarily by where the performance objective is formed and which decisions and information must be coordinated. They are not classified only by physical location or sampling rate. The same electric motor can support component-level torque production, vehicle-level yaw-moment generation, and mobility-system-level cooperative energy-efficient driving.

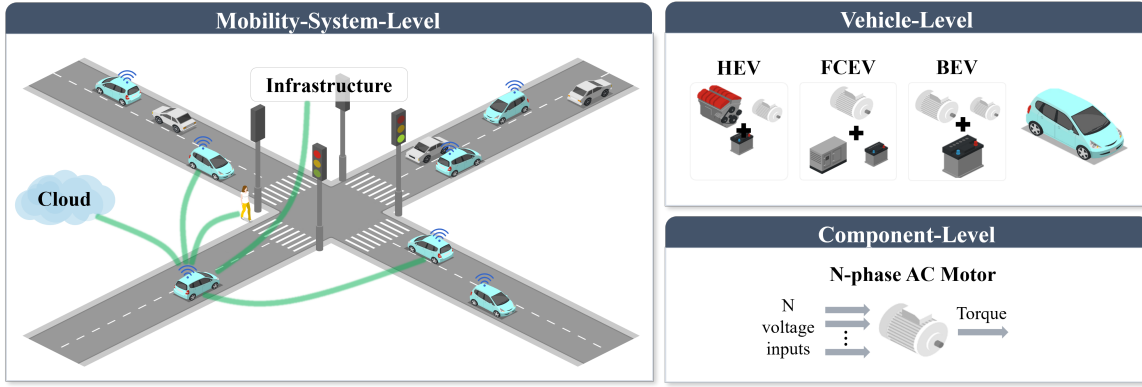


Figure 1.4: CAEV operation viewed at mobility-system, vehicle, and component levels. HEV, FCEV, and BEV denote hybrid electric, fuel-cell electric, and battery electric vehicles, respectively.

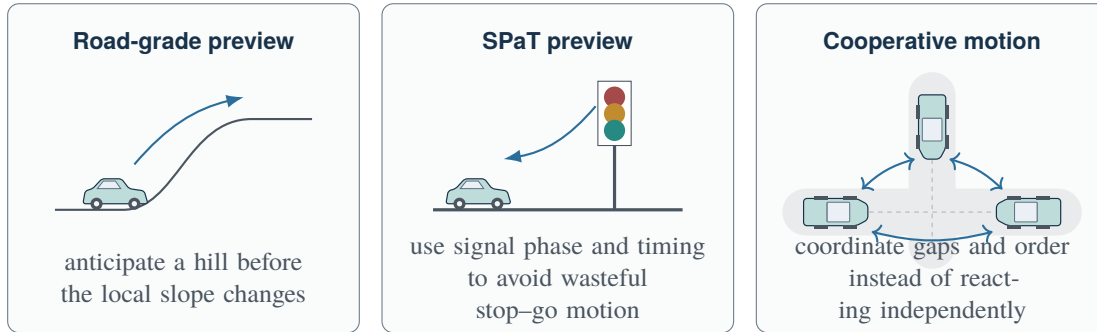


Figure 1.5: Schematic examples of mobility-system control enabled by preview and coordination: road-grade preview, SPaT preview, and cooperative motion.

Mobility-system level: use information and decisions beyond one local vehicle.

A mobility-system-level problem is defined by an information or decision scope that extends beyond the local vehicle boundary. Route, road grade, signal phase and timing (SPaT), traffic, weather, charging availability, infrastructure, and other agents may change the decision. Connected and automated capabilities are used actively, while electrification makes the energy consequence especially visible. Representative problems include predictive eco-driving and energy management, cooperative car following, platoon formation, merging, and intersection coordination.

The scale of the opportunity can be substantial, but numerical percentages must be read as scenario-dependent upper-end examples rather than transferable guarantees. The synthesis in Sciarretta and Vahidi (2020) reports examples on the order summarized in Table 1.2. The values are not additive and depend on baseline, penetration rate, traffic, vehicle, and control assumptions.

The three mechanisms are visualized in Figure 1.5. In each case, the essential change is preview or coordination: the controller acts on information that is not available from the vehicle's instantaneous local motion alone.

Table 1.2: Illustrative upper-end energy-saving opportunities enabled by preview and cooperation, synthesized from Sciarretta and Vahidi (2020).

Representative capability	Indicative upper end	What the controller gains
Eco-driving with preview of upcoming hills	up to 10%	Road-grade preview lets the vehicle trade speed and propulsion effort before the slope.
Eco-driving with SPaT preview	up to 30%	Signal timing lets the vehicle reduce unnecessary stops, acceleration, and braking.
Collaborative following and lane selection or cooperative intersection passage	up to 50%	Vehicle-to-vehicle (V2V) and vehicle-to-infrastructure (V2I) information enables several vehicles to coordinate gaps, order, and motion rather than reacting independently.

A recent Hyundai Motor Group service shows a practical information layer: live signal state and remaining time are delivered to an in-vehicle navigation system (Hyundai Motor Group 2025). This is not, by itself, proof of an energy-optimal speed controller. It shows that information once external to the automobile can become available for future preview-based decisions. Similarly, this [linked platooning and cooperative-lane demonstration](#) provides a visual example of vehicles changing gaps and group motion through connectivity and automation.

Vehicle level: coordinate actuation to shape the motion of the whole vehicle.

The vehicle-level boundary contains the complete vehicle. Driver or automated driving commands enter the controller, and onboard propulsion, braking, steering, and suspension actuators are coordinated to produce the intended longitudinal, lateral, and vertical response. Automation and electrification are particularly important: automation supplies desired motion and feedback decisions, while distributed electric drive, independent steering, and active suspension create redundant ways to generate forces and moments. Integrated chassis control or integrated vehicle motion control is the broad example (Mazzilli et al. 2021). The [linked industry demonstration](#) illustrates the idea of one supervisory vehicle-dynamics logic coordinating several active systems.

Component level: realize fast physical actuation and energy conversion.

The component level concerns a device and the variables required to make it deliver the requested physical action. In an electrified vehicle, these devices include the battery, fuel cell, electric machine, inverter, DC/DC converter, brake actuator, and steering or suspension actuator. Their current, voltage, flux, switching, pressure, force, torque, temperature, and state of charge dynamics form the base on which electrification is implemented. Optimal torque control of a

synchronous machine is the recurring example in this book: voltage or switching commands must generate torque efficiently while respecting electrical, magnetic, thermal, and inverter limits.

These levels are connected. A mobility-system coordinator specifies a speed or arrival-time target; the vehicle controller converts it into tire-force and yaw-moment demands; component controllers realize the requested forces and torques. CASE capabilities enlarge both the information available for a decision and the number of ways in which that decision can be realized. Fully exploiting these capabilities requires using expanded information to anticipate future conditions and account for cross-level effects, while allocating additional actuation freedom to meet force demands and balance energy use, comfort, and safety. Optimal control provides a systematic framework for doing so by selecting admissible decisions to optimize performance subject to system dynamics and constraints. The next section formalizes this framework.

1.2 Representative OCPs Across CAEV Control Levels

Optimal control provides a common decision principle across the three levels: find an admissible input sequence or feedback law that minimizes a declared performance index subject to dynamics and constraints. The principle is common; the state, input, objective, horizon, information, and physical limits change with the level.

A first finite-horizon formulation.

For an introductory deterministic problem with given initial state x_0 , let x_k be the state, u_k the control, f_k the dynamics, g_k the stage cost, G the terminal cost, and N the number of control stages. A transparent first formulation is

$$\begin{aligned} \min_{u_0, \dots, u_{N-1}} \quad & \sum_{k=0}^{N-1} g_k(x_k, u_k) + G(x_N) \\ \text{s.t.} \quad & x_{k+1} = f_k(x_k, u_k), \\ & x_k \in \mathcal{X}_k, \quad k = 0, \dots, N, \\ & u_k \in \mathcal{U}_k(x_k), \quad k = 0, \dots, N-1. \end{aligned} \tag{1.1}$$

The sets $\mathcal{X}_k$ and $\mathcal{U}_k(x_k)$ collect state and admissible input constraints. This notation is reused in Chapter 2. The symbol g is used consistently for stage cost throughout the book. The terminal term $G(x_N)$ represents a final requirement or an approximation of cost beyond the explicit horizon.

Finite horizon is the gentler first formulation because trips, maneuvers, preview windows, and predictive controllers have visible endpoints. A longer or infinite horizon can account more directly for consequences that persist beyond one maneuver, including energy and traffic effects. Later in this chapter, a stationary discounted infinite-horizon problem provides the reusable Bellman reference central to the book. The two views are connected under the same stage-cost and discount convention: if the terminal term equals the exact optimal cost beyond stage N , the finite problem has the same initial optimal decision as its infinite-horizon continuation. For example, in an α -discounted formulation, the terminal contribution is scaled as $\alpha^N J^*(x_N)$.

Chapter 5 develops this terminal-cost connection and its approximate use in model predictive control (MPC).

If dynamics are stochastic or the state is not observed directly, the decision variable must be a feedback law rather than a precomputed open-loop sequence. Let $\mathcal{I}_k$ denote the information available at time k , such as current and past measurements, past inputs, commands, messages, and admitted context. Then the generalization is

$$\begin{aligned} \min_{\mu_0, \dots, \mu_{N-1}} \quad & \mathbb{E} \left[\sum_{k=0}^{N-1} g_k(x_k, u_k, w_k) + G(x_N) \right] \\ \text{s.t.} \quad & x_{k+1} = f_k(x_k, u_k, w_k), \quad u_k = \mu_k(\mathcal{I}_k), \\ & x_k \in \mathcal{X}_k, \quad k = 0, \dots, N, \\ & u_k \in \mathcal{U}_k(x_k), \quad k = 0, \dots, N-1. \end{aligned} \quad (1.2)$$

Here w_k is a disturbance and the expectation is taken over the declared initial condition and stochastic evolution. The minimization is over nonanticipative laws: μ_k may use only the information in $\mathcal{I}_k$. Unless stated otherwise, the displayed constraints are required to hold almost surely. In a fully observed Markov model, the current state is sufficient and one writes $u_k = \mu_k(x_k)$. The three examples below are deliberately deterministic, so the expectation is omitted. They isolate recurring decision structures rather than implementation-complete designs.

Mobility-system level: centralized multi-vehicle coordination at an on-ramp merge.

Consider a main road and one on-ramp that meet at a single merging point, as in Figure 1.6. All participants are connected and automated vehicles (CAVs); their lanes are fixed, and a central coordinator receives their position and speed. This removes mixed-traffic prediction and lane-change decisions so that the coordination idea is visible. The scenario is a simplified all-CAV version of the framework in Choi, Shin, et al. (2025). The formulation first schedules arrival times at the merging point and then computes vehicle trajectories that realize those times.

Let $\mathcal{N} = \{1, \dots, n\}$ be the set of CAVs in the control zone and let $r_i \in \{0, 1\}$ identify the main road or ramp used by vehicle i . For vehicle i , let $p_i(t)$ be longitudinal position measured relative to the merging point, $v_i(t)$ speed, and $u_i(t)$ commanded acceleration:

$$\begin{aligned} \dot{p}_i(t) &= v_i(t), & \dot{v}_i(t) &= u_i(t), \\ \underline{u} &\leq u_i(t) \leq \bar{u}, & 0 &\leq v_i(t) \leq \bar{v}. \end{aligned} \quad (1.3)$$

Vehicle i enters a control zone of length L_c at time t_i^{in} with the known initial conditions $p_i(t_i^{\text{in}}) = -L_c$ and $v_i(t_i^{\text{in}}) = v_i^{\text{in}}$. Its merging time T_i is defined by $p_i(T_i) = 0$. The desired arrival time

$$T_i^{\text{des}} := t_i^{\text{in}} + \frac{L_c}{\bar{v}} \quad (1.4)$$

is the time obtained by traversing the control zone at the maximum allowed speed. To keep the upper-level problem honest about dynamic feasibility, define the reachable arrival-time set

$$\mathcal{T}_i := \{T \geq t_i^{\text{in}} : \text{there exist } p_i(\cdot), v_i(\cdot), u_i(\cdot) \text{ satisfying Equation (1.3),} \\ p_i(t_i^{\text{in}}) = -L_c, \quad v_i(t_i^{\text{in}}) = v_i^{\text{in}}, \quad p_i(T) = 0\}. \quad (1.5)$$

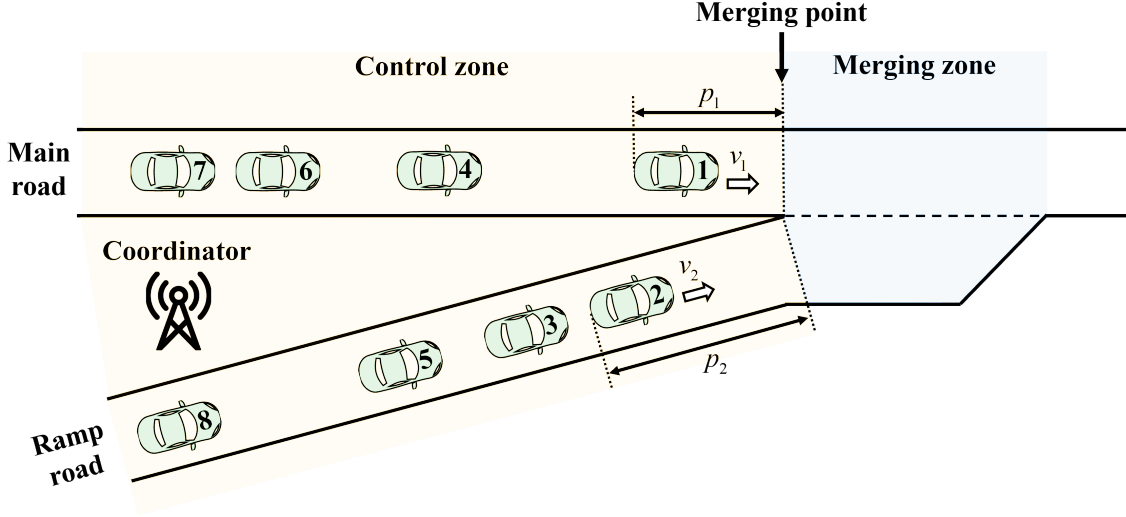


Figure 1.6: Mobility-system-level model for centralized multi-vehicle coordination at an on-ramp merge. Vehicles on the main road and ramp travel through a control zone toward a common merging zone, while a central coordinator uses their longitudinal positions p_i and speeds v_i to coordinate the merging process. The arrows associated with p_i indicate the remaining-distance magnitudes $-p_i$ under the signed-position convention used below. Reproduced from Figure 1 of Choi, Shin, et al. (2025).

Let $\text{pred}(i)$ denote the immediate predecessor of vehicle i on the same road, when one exists. A central scheduler determines the vehicles' arrival times from

$$\begin{aligned}
 \min_{\{T_i\}_{i \in \mathcal{N}}} \quad & \sum_{i \in \mathcal{N}} (T_i - T_i^{\text{des}})^2 \\
 \text{s.t.} \quad & T_i \in \mathcal{T}_i, \quad i \in \mathcal{N}, \\
 & |T_i - T_j| \geq T_{\text{lat}}, \quad i, j \in \mathcal{N}, \quad r_i \neq r_j, \\
 & T_i - T_{\text{pred}(i)} \geq T_{\text{rear}}, \quad i \in \mathcal{N} \text{ with } \text{pred}(i) \text{ defined.}
 \end{aligned} \tag{1.6}$$

Here $T_{\text{lat}} > 0$ separates vehicles arriving from conflicting roads, and $T_{\text{rear}} > 0$ preserves the order and headway of vehicles on the same road. The absolute-value constraint does not assume a cross-road crossing order in advance: the optimization implicitly selects which of two conflicting vehicles arrives first. This makes the constraint disjunctive and already exposes a combinatorial aspect of coordination.

Problem (1.6) determines arrival times. After the optimal time T_i^* has been assigned, the lower-level problem determines the acceleration input that realizes it with minimum control effort. In this sequential trajectory-generation view, the predecessor trajectory is treated as already determined and known over $[t_i^{\text{in}}, T_i^*]$, including any continuation beyond its own merging

time needed to evaluate spacing:

$$\begin{aligned}
 \min_{u_i(\cdot)} \quad & \frac{1}{2} \int_{t_i^{\text{in}}}^{T_i^*} u_i(t)^2 dt \\
 \text{s.t.} \quad & \text{dynamics and bounds in (1.3),} \\
 & p_i(t_i^{\text{in}}) = -L_c, \quad v_i(t_i^{\text{in}}) = v_i^{\text{in}}, \\
 & p_{\text{pred}(i)}(t) - p_i(t) \geq d_{\min} + T_{\min} v_i(t), \quad \text{if } \text{pred}(i) \text{ exists, } t \in [t_i^{\text{in}}, T_i^*], \\
 & p_i(T_i^*) = 0, \quad u_i(T_i^*) = 0.
 \end{aligned} \tag{1.7}$$

The spatial constraint uses standstill distance $d_{\min} > 0$ and time headway $T_{\min} > 0$. The upper-level reachable set checks individual dynamic feasibility. If the predecessor-dependent spacing constraint makes the lower-level problem infeasible, the schedule must be revised or the reachable set must be strengthened to include coupled trajectory feasibility. Even this simplified all-CAV problem couples vehicles through arrival order, time gaps, and dynamically feasible motion. Human-driven vehicles add uncertain reactions; lane changes add lateral motion and gap selection; and an intersection adds several conflict points. The state and decision space then grow while the information needed to predict other agents becomes less certain.

The continuous-time model is retained because arrival at a merge is naturally an event-time decision and because it matches the cited formulation. It is fully compatible with the discrete-time OCP above. Under a zero-order-hold input with sample time h ,

$$p_{i,k+1} = p_{i,k} + h v_{i,k} + \frac{h^2}{2} u_{i,k}, \quad v_{i,k+1} = v_{i,k} + h u_{i,k}, \tag{1.8}$$

which has exactly the state-update form in Equations (1.1) and (1.2). Continuous-time plant models and sampled decision models will therefore coexist throughout the book.

Vehicle level: integrated lateral and yaw-motion control.

A full integrated vehicle model can include longitudinal, lateral, vertical, wheel, and actuator dynamics. A reduced linear bicycle model isolates two essential lateral states: body sideslip angle β and yaw rate r . The model geometry is summarized in Figure 1.7. It provides a transparent entry point while retaining the coordination of two actuator families.

At a constant longitudinal speed $v_x > 0$, let ℓ_f and ℓ_r be the distances from the center of gravity to the front and rear axles, m the vehicle mass, and I_z the yaw inertia. Let δ_f be the front road-wheel steering angle. The direct yaw-moment command M_z is a generalized vehicle-level input that a lower-level allocator can realize through left-right differential drive or brake torques. Under the usual small-angle bicycle-model assumptions,

$$\begin{aligned}
 m v_x (\dot{\beta} + r) &= F_{yf} + F_{yr}, \\
 I_z \dot{r} &= \ell_f F_{yf} - \ell_r F_{yr} + M_z, \\
 F_{yf} &= C_f \left(\delta_f - \beta - \frac{\ell_f r}{v_x} \right), \quad F_{yr} = C_r \left(-\beta + \frac{\ell_r r}{v_x} \right),
 \end{aligned} \tag{1.9}$$

where F_{yf} and F_{yr} are the front and rear axle lateral tire forces, and C_f and C_r are the corresponding axle cornering stiffnesses. After sampling, $x_k = [\beta_k \ r_k]^\top$ and $u_k = [\delta_{f,k} \ M_{z,k}]^\top$ satisfy

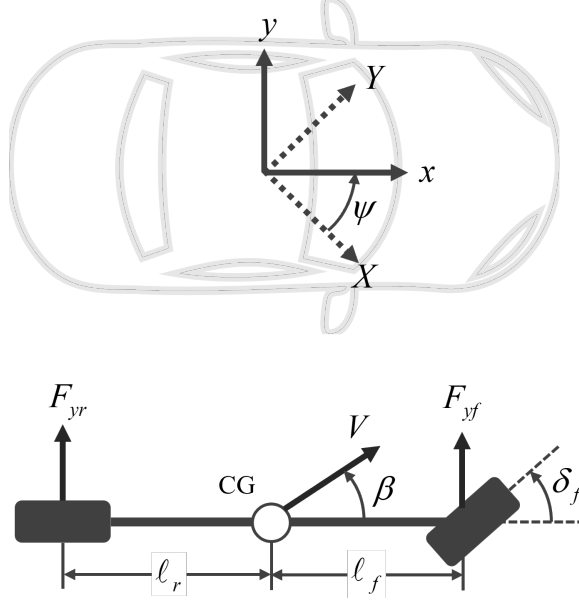


Figure 1.7: Vehicle-level model for lateral and yaw dynamics. The upper diagram defines the inertial coordinates (X, Y) and vehicle-fixed coordinates (x, y) , while the lower diagram gives the single-track representation and the center-of-gravity location specified by the axle distances ℓ_f and ℓ_r . The associated integrated-motion control problem is formulated separately in the text.

$x_{k+1} = Ax_k + Bu_k$, where A and B are the sampled system matrices. A representative finite-horizon problem is

$$\begin{aligned} \min_{u_0, \dots, u_{N-1}} \quad & \sum_{k=0}^{N-1} (q_\beta \beta_k^2 + q_r (r_k - r_k^{\text{ref}})^2 + q_\delta \delta_{f,k}^2 + q_M M_{z,k}^2) + G(x_N) \\ \text{s.t.} \quad & x_{k+1} = Ax_k + Bu_k, \quad |\delta_{f,k}| \leq \bar{\delta}_f, \quad |M_{z,k}| \leq \bar{M}_z. \end{aligned} \quad (1.10)$$

Here r_k^{ref} is the reference yaw rate. The weights $q_\beta, q_r, q_\delta, q_M \geq 0$ trade stability, yaw-response tracking, and actuator use. This simple model already shows integrated action: front-wheel steering shapes lateral tire forces while differential wheel torques supply an additional yaw moment; both contribute to the same vehicle motion. The simple bound on M_z represents an outer approximation of the state-, friction-, and actuator-dependent moment that the allocator can actually produce.

The real vehicle is harder. Tire lateral force also depends on longitudinal force and vertical load; braking or propulsion can consume the same friction capacity needed for steering. Roll and load transfer change axle and wheel forces. Actuator bandwidth, saturation, tire condition, road friction, and payload vary. Moreover, tuning is judged not only by β and r : roll response, comfort, naturalness, and driver-perceived vehicle character often enter validation. Thus the compact state and quadratic cost are a transparent starting point, not the end of integrated vehicle motion control (Rajamani 2012; Mazzilli et al. 2021).

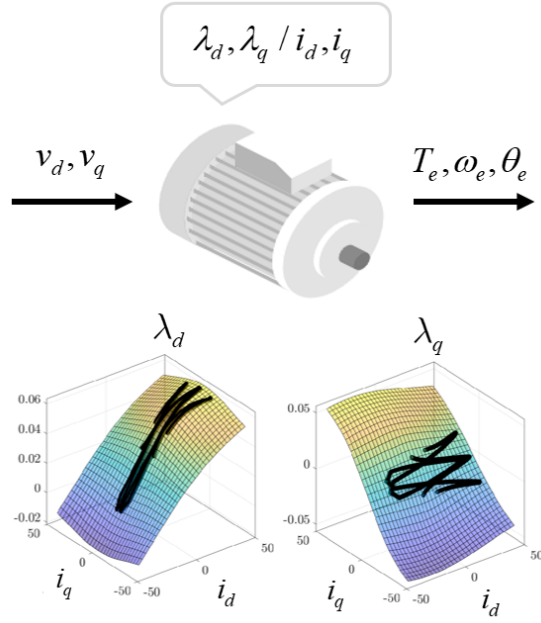


Figure 1.8: Component-level synchronous-machine model relating the dq -axis voltages v_d, v_q , currents i_d, i_q , flux linkages λ_d, λ_q , electromagnetic torque T_e , electrical angular speed ω_e , and electrical rotor angle θ_e . The lower panels show the nonlinear flux-linkage maps $\lambda_d(i_d, i_q)$ and $\lambda_q(i_d, i_q)$.

Component level: optimal electric-machine torque control.

For a synchronous machine, let i_d, i_q be rotating-frame currents, v_d, v_q voltages, λ_d, λ_q flux linkages, R_s stator resistance, ω_e electrical speed, and n_p the number of pole pairs. A general dq representation is

$$\begin{aligned} v_d &= R_s i_d + \frac{d\lambda_d}{dt} - \omega_e \lambda_q, & v_q &= R_s i_q + \frac{d\lambda_q}{dt} + \omega_e \lambda_d, \\ T_e &= \frac{3n_p}{2} (\lambda_d i_q - \lambda_q i_d). \end{aligned} \quad (1.11)$$

The nonlinear flux map may be written $[\lambda_d \ \lambda_q]^\top = \Lambda(i_d, i_q; c_m)$, where c_m collects motor operating quantities such as temperature. Flux linkage is generally not measured directly at the control rate, whereas the currents are measured or reconstructed. Assuming that Λ is known or estimated, it can be substituted into Equation (1.11); the resulting current dynamics can then be discretized with $x_k = [i_{d,k} \ i_{q,k}]^\top$, $u_k = [v_{d,k} \ v_{q,k}]^\top$, and $x_{k+1} = f(x_k, u_k; c_{m,k})$. Figure 1.8 summarizes the electrical, magnetic, and mechanical variables used in this model.

One predictive torque-control problem is

$$\begin{aligned}
& \min_{u_0, \dots, u_{N-1}} \sum_{k=0}^{N-1} \left[q_T (T_{e,k+1} - T_k^{\text{cmd}})^2 + q_{\text{loss}} P_{\text{loss},k+1} \right] + G(x_N) \\
& \text{s.t. } x_{k+1} = f(x_k, u_k; c_{\text{m},k}), \\
& \|x_{k+1}\|_2 \leq I_{\text{max}}, \quad \|u_k\|_2 \leq V_{\text{max}},
\end{aligned} \tag{1.12}$$

where the current command T_k^{cmd} is tracked by the next predicted torque $T_{e,k+1}$, $P_{\text{loss},k+1}$ is a declared copper, iron, inverter, or combined loss model, and $q_T, q_{\text{loss}} \geq 0$. Current and voltage limits appear directly; thermal and switching constraints can be added when their dynamics matter. This formulation is consistent with constrained predictive torque control, including one-step implementations (Choi, Kim, et al. 2025).

The component problem is compact in dimension but not automatically easy. Magnetic saturation, cross-coupling, loss, temperature, inverter nonlinearity, and parameter drift make f and T_e uncertain. In a continuous-control-set (CCS) design, the optimizer chooses a voltage vector from a continuous admissible region. In a finite-control-set (FCS) design, it chooses one of the inverter's discrete switching states. The latter is attractive because the switch command can be applied directly, but the number of switching sequences grows rapidly with prediction horizon. In the FCS version of Equation (1.12), the continuous voltage constraint is replaced by $u_k \in \mathcal{U}_{\text{sw}}$, where $\mathcal{U}_{\text{sw}}$ is the finite admissible voltage-vector set generated by the inverter switching states. Both cases must fit nonlinear prediction and constraint handling into a short electrical sampling period, so longer-run efficiency competes with a tight online computation budget.

1.3 Why, Where, and When Learning-Based Control Enters

The three examples were intentionally simple, yet each already exposes a gap between writing an optimal control problem and deploying its solution. Table 1.3 groups that gap into two recurring difficulties. The first is computational: the relevant decision may be high-dimensional, constrained, hybrid, or required faster than a direct optimization can be completed. The second is informational: the dynamics, state, cost, constraints, or future disturbances needed by the optimizer may be uncertain, time-varying, or only partially observed.

These difficulties identify where tractable approximations and information extracted from data can help. They may compress a long optimization into a value or policy representation, approximate costly expectations or action searches, or supply a transition model or decision state that first-principles models and sensors do not provide adequately. Their usefulness is judged by the resulting closed loop under the original physical objective, constraints, and safety requirements; an approximation does not make those requirements disappear.

Table 1.3: Why direct optimal control becomes difficult in the three examples.

Representative OCP	Why direct solution becomes difficult
Centralized multi-vehicle coordination at an on-ramp merge	Computation: joint trajectories, disjunctive arrival order, collision constraints, and short replanning deadlines. Information: future arrivals, communication, human-driver response, intentions, and changing traffic composition.
Integrated vehicle motion control using front-wheel steering and direct yaw moment	Computation: nonlinear coupled dynamics, actuator allocation, and broad operating-domain calibration. Information: tire forces, friction, load, actuator state, and imperfect numerical proxies for desired vehicle character.
Optimal electric-machine torque control	Computation: fast nonlinear constrained optimization and discrete search under FCS operation. Information: flux, loss, resistance, temperature, inverter behavior, and unmeasured electrical states.

Why Bellman optimality is the organizing principle.

The roles just described become precise once the optimal control problem is expressed as a feedback recursion. Bellman’s principle of optimality provides that recursion: it separates a multistage decision into its immediate cost and optimal continuation cost, thereby exposing two reusable objects—a value function and a feedback law. Dynamic programming (DP) is the family of recursive solution methods that turns this decomposition into computations. Its approximate and sample-based variants later provide the bridge to reinforcement learning (RL).

Bellman’s framework is not the only route to optimal control. Minimum-principle methods and direct transcription remain indispensable alternatives. Appendix D gives a compact map of these routes and distinguishes open-loop solutions from feedback laws. Bellman’s principle is used here because it places exact DP, approximate DP, rollout, MPC, and control-oriented RL within a common feedback framework.

The finite-horizon problems above have stage-dependent Bellman recursions. The stationary discounted form below serves as the reusable reference for the exact, approximate, and online-improvement methods developed later.

A compact Bellman recursion requires the decision problem to be expressed in an appropriate state. The reference problem below is *stationary*: its dynamics, stage cost, admissible-input set, and disturbance law do not change explicitly with the stage index. Its state must also be *Markov*: once the current state x and control u are given, the conditional distribution of the next state and one-stage cost must not depend on the earlier history. This is a requirement on how the decision state is constructed, not a guarantee that a raw sensor vector is already sufficient. If the measurements omit decision-relevant memory, x must be augmented with quantities such as time, route or signal phase, command, temperature, estimator state, or a belief distribution so that the Bellman recursion is valid.

Let w denote a random disturbance, let $x^+ = f(x, u, w)$ be the successor state, let $g(x, u, w)$ be one-stage cost, and let $\mathcal{U}(x)$ be the admissible-control set. For a stationary feedback law μ , define its discounted cost and the optimal cost by

$$\begin{aligned} J_\mu(x) &:= \mathbb{E}_\mu \left[\sum_{k=0}^{\infty} \alpha^k g(x_k, \mu(x_k), w_k) \mid x_0 = x \right], \\ J^*(x) &:= \inf_{\mu} J_\mu(x), \quad 0 < \alpha < 1, \end{aligned} \tag{1.13}$$

where α is the discount factor. The infinite-horizon reference is useful because many mobility controllers operate repeatedly and their energy, traffic, or thermal consequences persist beyond one maneuver. It does not exclude finite missions or finite online lookahead.

Under the standard regularity conditions developed in Chapter 2, the principle of optimality gives

$$\begin{aligned} J^*(x) &= \min_{u \in \mathcal{U}(x)} \mathbb{E}_{w|x,u} [g(x, u, w) + \alpha J^*(f(x, u, w))], \\ \mu^*(x) &\in \arg \min_{u \in \mathcal{U}(x)} \mathbb{E}_{w|x,u} [g(x, u, w) + \alpha J^*(f(x, u, w))]. \end{aligned} \tag{1.14}$$

The first line characterizes the optimal value J^* ; the second obtains an optimal feedback law μ^* . DP supplies recursive algorithms for solving this Bellman relation when its state-space representation and computations are tractable. Chapter 2 develops those algorithms and their assumptions.

What this book calls learning-based control.

The Bellman relation identifies the desired value and feedback law, but obtaining them exactly can be impractical. Exact DP may require representing and updating a value over a large or continuous state space, evaluating expectations over uncertainty, and minimizing over admissible actions at many states. It also requires a sufficient decision state together with transition, cost, and disturbance information. These requirements reproduce the computational and information difficulties summarized in Table 1.3 and motivate approximation.

In this book, *approximation* is the broader concept. It replaces an exact decision-relevant object, model, or computation by a tractable surrogate. Examples include a fitted value or policy, a learned model or state estimator, a simplified model or control problem, a numerical or sample-based estimate of an expectation, and a restricted or early-terminated action minimization. Data-based *learning* is one route within this broader class: measured, simulated, demonstrated, or otherwise generated data are used to construct or revise an approximate value, policy, model, or state estimator. Thus a learned object is ordinarily an approximation of the desired object, whereas an approximation need not be obtained by fitting data.

Accordingly, this book uses *learning-based control* as a broad, control-oriented framework built around approximation, including approximations constructed or revised through data-based learning. The name does not imply that every approximation is fitted from data. It emphasizes a control architecture in which tractable surrogates—and learning when data are used—are intended to make feedback design and online improvement feasible while retaining an explicit connection to the physical objective, system, constraints, and improvement mechanism.

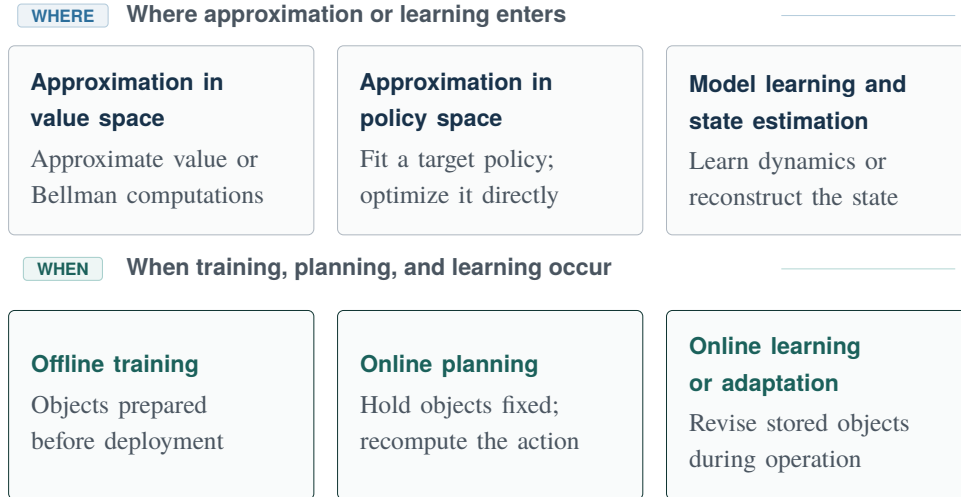


Figure 1.9: Two complementary design axes for learning-based control. The upper row shows where approximation or learning enters; the lower row distinguishes offline training, online planning with fixed decision objects, and online learning or adaptation. The two axes can be combined independently.

Approximation or learning may enter at three nonexclusive locations: value information and its Bellman computations, the feedback law—called a policy in RL—and the model or state estimator. These lead to the three method families used throughout the book: approximation in value space, approximation in policy space, and model learning or state estimation.

Two independent axes organize these choices. The *where* axis identifies which of the three locations is approximated or learned. The *when* axis asks whether the objects used by the controller are prepared before deployment, held fixed while a current-state planning problem recomputes the action, or revised from new operational data. These cases are called offline training, online planning, and online learning or adaptation, respectively.

The next three parts follow the method families in the upper row of Figure 1.9. The timing distinction in the lower row is developed after those families.

Approximation in value space.

The Bellman expression in Equation (1.14) exposes three places where exact computation may be replaced by an approximation: the future cost-to-go (or continuation value), the expectation over w conditional on the current (x, u) , and the minimization over admissible actions. These sites are marked in Figure 1.10. The name *approximation in value space* reflects the use of value information as the organizing object; it does not mean that only the value representation may be approximated, nor does it exclude deriving or improving a policy from that value. Here, *continuation value* is another name for the future cost-to-go, and *conditional* means that the current x and candidate u are held fixed while the possible values of w are averaged. Let $\tilde{J}$ denote a tractable approximation of the exact value J^* .

Cost-to-go approximation. The exact value $J^*(x)$ may be replaced by a fitted representation

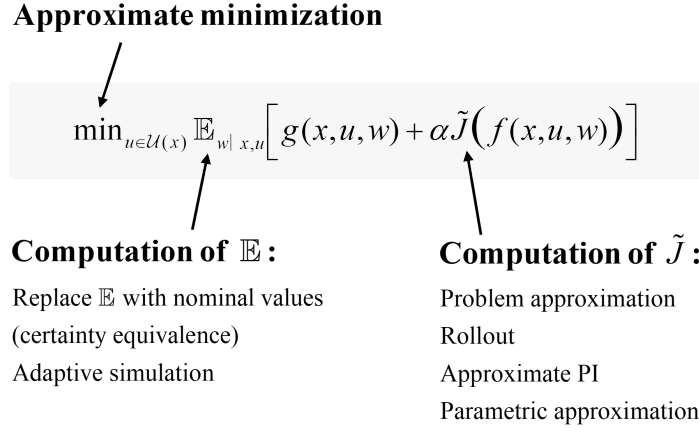


Figure 1.10: Three approximation sites in a Bellman-based decision: action minimization, the expectation over uncertainty, and the future cost-to-go. The three approximations need not all be used together.

$\tilde{J}(x; r_J)$, where r_J denotes adjustable coefficients. An action-dependent value, or *Q-factor*, may similarly be represented by $\tilde{Q}(x, u; r_Q)$; Chapter 2 gives its formal definition, and Chapter 3 develops the parameterization. A value or Q-factor approximator used to evaluate decisions is called a *critic*. Its targets may be generated by approximate value iteration (VI), approximate policy iteration (PI), Monte Carlo (MC), or temporal-difference (TD) policy evaluation. A learned Q-factor can support $\tilde{\mu}_Q(x) \in \arg \min_{u \in \mathcal{U}(x)} \tilde{Q}(x, u; r_Q)$ without an explicit transition model at action selection.

Not every cost-to-go approximation is a fitted parametric function. In *problem approximation*, $\tilde{J}$ is the optimal or nearly optimal value of a simpler control problem obtained, for example, by reducing the state, exploiting decomposition, relaxing selected constraints, or simplifying uncertainty. In *rollout*, the tail cost needed for the current decision is estimated by simulating a base policy, possibly over a truncated horizon followed by a terminal approximation. Rollout is therefore a value-space method according to what supplies its tail value and an online planning method when that value is generated at the current state. Chapter 4 develops rollout and the other principal value-space methods; problem approximation is retained here as a complementary design pattern.

Expected-value approximation. Even when a value representation is available, the expectation over w may be unavailable analytically or too costly to evaluate repeatedly. It can be approximated by a nominal disturbance, numerical quadrature, a sample average, transitions drawn from a simulator, or predictions from a learned transition or disturbance model. Replacing uncertainty by a nominal value is often called a certainty-equivalent approximation. Numerical integration and sampling are computational approximations, not necessarily data-based learning. They become part of data-based learning when samples are used to construct or revise a distribution, model, critic, policy, or other decision-relevant object.

When simulation is expensive, *adaptive simulation* allocates additional samples selectively to disturbance or action branches that are uncertain or important for the current decision instead

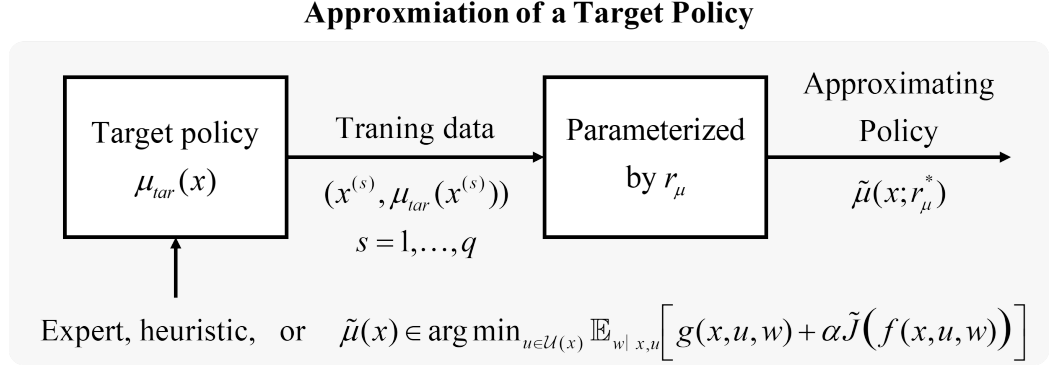


Figure 1.11: Policy-space approximation by fitting target actions supplied by an expert, heuristic, classical controller, or value-based improvement procedure.

of sampling all branches uniformly. Because this allocation is guided by comparisons among candidate actions, it may approximate the expectation and minimization jointly. Monte Carlo tree search is a multistep example; its timing classification is discussed below.

Simplified action minimization. The action minimization may be approximated by discretizing or restricting the candidate actions, terminating a numerical optimizer early, using sampling or random search, or letting an actor propose promising actions. The last choice connects value-space and policy-space approximation: the critic supplies the criterion, while a learned policy approximates part of the minimization. This decomposition follows the approximate-DP viewpoint developed in Bertsekas (2019) and Bertsekas (2025).

The advantage is structural. A value summarizes consequences beyond the immediate decision and can be reused from many current states. A critic may be used directly for action selection or later as a tail value in online planning. The limitations are equally important: fitting a useful value over a large state space can be difficult, Bellman targets may require a model or many transitions, and a small average prediction error does not necessarily preserve the action that minimizes the Bellman expression. Chapters 3–5 develop these points and show why online lookahead can reduce the effect of critic error.

Approximation in policy space.

This route represents a deterministic feedback law, or policy in DP/RL language, directly as $u = \tilde{\mu}(x; r_\mu)$. When action randomization is explicit, this book uses $u \sim \pi(\cdot | x)$ for a stochastic policy. A parameterized policy used in this way is called an *actor*. Two constructions should be distinguished because their data and optimization requirements are different.

Fitting a target policy. An expert, heuristic, classical controller, or previously computed policy can supply desired actions at selected states. Importantly, the target may also be the greedy or lookahead policy obtained from the value-space route above. Denote this target rule by μ_{tar} ; at a sampled state $x^{(s)}$, it supplies $u^{(s)} = \mu_{\text{tar}}(x^{(s)})$. Thus target-policy fitting can compress a costly value-based decision into a policy that is inexpensive to evaluate, as illustrated in Figure 1.11.

Direct Policy Optimization

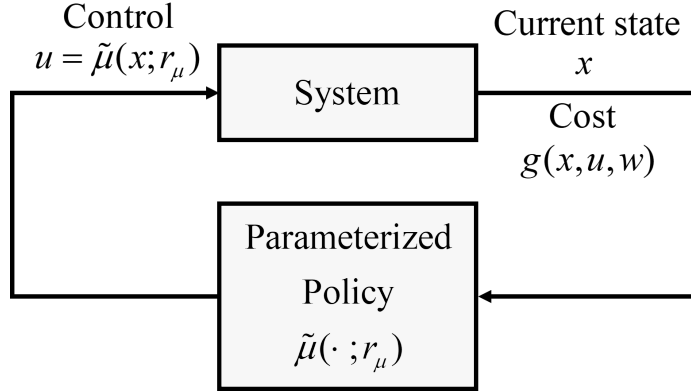


Figure 1.12: Direct policy optimization adjusts a parameterized policy using the closed-loop performance generated through interaction with the system.

Given q continuous-action targets $\{(x^{(s)}, u^{(s)})\}_{s=1}^q$, an actor may be fitted by

$$r_\mu^* \in \arg \min_{r_\mu} \sum_{s=1}^q \|u^{(s)} - \tilde{\mu}(x^{(s)}; r_\mu)\|_2^2. \quad (1.15)$$

For a finite action set, the analogous actor is normally a classifier or a probability model rather than a regressor.

Direct policy optimization. Instead of supplying target actions, one can adjust the policy using the closed-loop performance produced by the policy itself. Figure 1.12 shows this second route: the actor acts on the current state, the system generates subsequent states and costs, and those trajectories define the policy objective.

One representative objective is

$$r_\mu^* \in \arg \min_{r_\mu} \mathbb{E}_{x_0 \sim \rho_0} [J_{\tilde{\mu}(\cdot; r_\mu)}(x_0)], \quad (1.16)$$

where ρ_0 is a declared initial-state distribution. Gradient-free policy search, policy-gradient methods, and actor–critic algorithms provide different ways to address this objective. In an actor–critic method, the actor is the policy approximator and the critic is the accompanying value or Q-factor approximator; the combination therefore spans policy and value space.

Direct policy approximation avoids a numerical minimization when the actor is executed alone and accommodates continuous actions naturally. It must, however, be trained over the states it will visit, may reach only a local solution, and can conceal why an action is good or how a changed constraint should be handled. Chapter 6 develops target-policy fitting and direct policy optimization, including gradient-free search, policy-gradient methods, Proximal Policy Optimization (PPO), Soft Actor–Critic (SAC), and their combinations with a value-space critic and online improvement.

Model learning and state estimation.

The transition model and the decision state form the third family. A learned model can supply the transition or disturbance information needed to evaluate a Bellman expectation and to predict within online planning. A learned state estimator can supply the decision state used by a critic, actor, or planner. A full learned deterministic model has the form $x_{k+1} \approx \hat{f}(x_k, u_k; r_f)$, where r_f is the learned full-model parameter vector. When useful physics is already available, a residual representation is often more transparent:

$$x_{k+1} = f_{\text{nom}}(x_k, u_k) + \hat{h}(x_k, u_k; r_h) + \varepsilon_k, \quad (1.17)$$

where f_{nom} is the nominal model, $\hat{h}$ is the learned model discrepancy, and ε_k is the remaining prediction error. The learned object may instead be a probabilistic transition, disturbance distribution, constraint boundary, or stage cost.

For example, q transition samples $\{(x^{(s)}, u^{(s)}, x_+^{(s)})\}_{s=1}^q$, where $x_+^{(s)}$ is the successor state of sample s , support the residual fit

$$r_h^* \in \arg \min_{r_h} \sum_{s=1}^q \|x_+^{(s)} - f_{\text{nom}}(x^{(s)}, u^{(s)}) - \hat{h}(x^{(s)}, u^{(s)}; r_h)\|_2^2. \quad (1.18)$$

This basic supervised objective is only a starting point: closed-loop data coverage, multistep prediction, uncertainty, and physical consistency determine whether the fitted model is useful for control.

When x_k is not measured, an observer or estimator constructs $\hat{x}_k$ from measurements and inputs. Identification and estimation are coupled because the state trajectory and the dynamics must explain the same data. Model learning predicts how the environment evolves; state estimation reconstructs the current decision state from measurement and input history. They are distinct tasks but may be trained and implemented jointly. A classical fixed-model observer is not itself data-based learning, but it belongs to the state-estimation branch when it supplies an approximate decision state. Learning or adaptation enters when a model, correction map, state representation, or parameter is fitted from data or updated online. A learned model can then support online lookahead and MPC, synthetic rollouts for value or policy training, uncertainty-aware prediction, and state reconstruction. Chapter 7 begins from full, residual, deterministic, and probabilistic model choices before treating closed-loop data, uncertainty, distribution shift, and continual model learning.

The three families can be combined.

The upper row of Figure 1.9 is not a choice among mutually exclusive algorithms. A value-space method may generate the target policy used to train an actor. An actor may propose or warm-start actions while a critic evaluates their future consequences. A learned model may supply predictions to a Bellman backup or online lookahead, while a state estimator supplies the state used by every other block. Actor–critic methods span value and policy space, and learned-model planning spans model and value or policy space. Chapters 4–7 develop these combinations and distinguish the role of each decision-relevant object.

When training, planning, and learning occur.

The object used by the controller and the time at which it is prepared, used, or revised are independent axes. Here *online planning* has a deliberately limited meaning: a model or simulator is used at the current state to recompute an action or short plan while stored critics, actors, models, and estimators remain fixed. Online learning or adaptation instead revises at least one stored object using new operational data. Table 1.4 establishes the three terms used throughout the book.

Table 1.4: Offline training, online planning, and online learning are distinct.

Mode	What happens to the stored object?	Representative use
Offline training	An approximate or learned value, policy, model, or estimator is prepared before deployment and then stored.	fitted value iteration, approximate policy iteration, critic fitting, actor imitation or policy search, and system identification
Online planning	Stored decision objects remain unchanged while a model or simulator is used to recompute the current action or a short plan.	one- or multistep lookahead, MPC, rollout, and Monte Carlo tree search
Online learning or adaptation	Operational data revise a stored value, policy, model, estimator, or controller.	recursive identification, adaptive estimation, continual model learning, and online critic or actor updates

Offline training. Data available before deployment may be used to construct a critic, imitate a target policy, optimize an actor, or identify a model. At deployment, the stored object can be evaluated directly or supplied to an online planner. The fact that a controller uses a learned object online does not by itself make the controller an online-learning method. A data-free approximation may likewise be prepared before deployment; in that case, *offline preparation* is the more literal description, while the table retains the familiar shorthand *offline training* for the timing category.

Online planning. A representative way to combine an offline critic with current-state planning is the deterministic ℓ -step lookahead law. Let the critic provide the terminal evaluator $\hat{J}_T(\cdot) := \tilde{J}(\cdot; r_J)$. Then

$$\begin{aligned}
\mu_\ell(x) &= u_0^*, \\
(u_0^*, \dots, u_{\ell-1}^*) &\in \arg \min_{u_0, \dots, u_{\ell-1}} \sum_{j=0}^{\ell-1} \alpha^j g(x_j, u_j) + \alpha^\ell \hat{J}_T(x_\ell) \\
\text{s.t. } x_0 &= x, \quad x_{j+1} = f(x_j, u_j), \quad u_j \in \mathcal{U}(x_j), \quad j = 0, \dots, \ell-1.
\end{aligned} \tag{1.19}$$

At the current state x , the planner holds $\hat{J}_T$ and the prediction model fixed, solves the finite lookahead problem, applies only the first action u_0^* , and repeats at the next state. A stochastic extension optimizes nonanticipative feedback laws over the lookahead tree rather than a single open-loop input sequence. For $\ell = 1$, the law is a one-step lookahead decision with an approximate tail value; for $\ell > 1$, it is multistep lookahead. Resolving the constrained problem at each new state gives the structure of MPC with an approximate terminal cost. Rollout uses a base policy to estimate or complete the tail, while an actor can warm-start or guide the search. In all cases, the current decision changes with the current state even though the stored decision objects do not.

Monte Carlo tree search (MCTS) is an online-planning method when it is run at decision time from the current state, uses a model or generative simulator to construct a selective lookahead tree, and adaptively allocates simulations to promising branches. Although MCTS updates temporary statistics within the search tree, it need not revise any persistent value function, policy, or model. If search experience is retained and used to update such persistent objects, the architecture combines online planning with learning.

Evaluating a stored actor is online execution rather than online planning. Likewise, minimizing a stored Q-factor over the current action is single-state action selection; it becomes forward planning only when a transition model or simulator is used to expand future consequences explicitly.

Online learning or adaptation. Here new measurements, costs, or transitions obtained during operation revise a critic, actor, model, estimator, or structured controller, and subsequent decisions use the revised object. The data are therefore coupled to the controller: today's decisions determine which states and actions generate the data used for tomorrow's update. Coverage, excitation or exploration, distribution shift, computation time, stability, and safe update acceptance become part of the control problem. The term *adaptation* often denotes a structured recursive update of a model, estimator, or controller, whereas *online learning* also includes broader data-driven revisions. They are grouped on this timing axis because both change a stored decision-relevant object during operation.

The where and when axes yield coherent combinations. One can train an actor offline and evaluate it directly, fit a critic offline and improve the current action by lookahead, improve a base policy by rollout, or revise a critic, actor, model, or state estimator from operational transitions. MPC or rollout can therefore improve an offline object without learning online; conversely, a model can be learned online while the optimization structure remains fixed. In every case, training or prediction loss is only an intermediate diagnostic: the resulting controller must ultimately be evaluated by closed-loop cost, constraint satisfaction, computation time, and behavior under shifted conditions relative to meaningful baselines.

1.4 A Control–RL Terminology Bridge

Control and RL communities often describe closely related sequential decision structures using different symbols and vocabulary. The working translation in Table 1.5 is adopted throughout this book. The main exposition uses the cost-minimization language of control and DP, with terminology notes providing corresponding terms commonly used in RL. These correspondences

are contextual; they should not be read as claims that every pair of terms is interchangeable in every setting.

Table 1.5: A working translation between control/DP and RL language.

Control/DP language	RL language
system	environment
controller (or decision maker)	agent
state x , control u	state s , action a
feedback law, $u = \mu(x)$	policy, π : deterministic, $a = \pi(s)$; stochastic, $a \sim \pi(\cdot s)$
stage cost g ; minimize cumulative cost	reward $r = -g$; maximize return
cost-to-go $J_\mu(x)$	state value $V^\pi(s) = -J_\mu(x)$
cost Q-factor $Q_\mu(x, u)$	action value $Q^\pi(s, a) = -Q_\mu(x, u)$
policy evaluation	prediction
current-state lookahead using a model or simulator	planning
system identification	model learning
cost-minimizing greedy improvement with respect to J or Q	reward-maximizing greedy improvement with respect to V or Q
terminating or episodic task	episodic task
nonterminating repeated task	continuing task
Bellman operator applied at one state	backup

The planning row uses the limited meaning introduced in Section 1.3: a model or simulator expands future consequences from the current state while persistent decision objects remain fixed. It does not equate every explicit-model DP computation with online planning.

The fourth row uses the conventional notation of each community. In control, a feedback law is commonly written as the deterministic mapping $u = \mu(x)$. In RL, the term *policy* is broader: the same symbol π may denote a deterministic policy, $a = \pi(s)$, or a stochastic policy, $a \sim \pi(\cdot | s)$. In the latter case, $\pi(a | s)$ is a probability mass function for discrete actions or a probability density for continuous actions. A stochastic policy is therefore not merely a deterministic feedback law applied to a stochastic system; it explicitly specifies a distribution over actions. Outside terminology bridges, this book normally uses μ for deterministic feedback laws and $\pi(\cdot | x)$ when policy randomization is explicit.

Some of the preceding terms will be introduced formally in future chapters; they are collected here as a reference, and no prior familiarity is assumed. To keep the derivations consistent across physical mobility examples, the main exposition retains x, u, w, f, g, J , and α . These symbols make the system, disturbance, cost, and feedback decision explicit in deterministic as well as stochastic models and in continuous as well as discrete spaces. The symbols s, a, r , and γ are used primarily when a named RL algorithm or an external RL result is stated. This is a change

of language, not a change of the underlying sequential decision problem. A sensor observation is not automatically a Markov state, so state and observation are distinguished when partial observability matters.

The most important translation is the sign convention. If $r_k = -g_k$ and $\gamma = \alpha$, then maximizing $\mathbb{E}[\sum_k \gamma^k r_k]$ is equivalent to minimizing $\mathbb{E}[\sum_k \alpha^k g_k]$. Consequently, an RL maximization over Q becomes a control minimization over the corresponding cost Q-factor. A positive “advantage” in reward language indicates an action better than the baseline; with cost advantages the favorable sign is reversed unless the definition itself includes a minus sign. Each chapter states the convention before using the quantity.

DP and RL are distinguished here more by emphasis than by a sharp boundary. For the sequential control problems in this book, DP begins with Bellman’s structural recursion and asks how value and policy relations can be evaluated and improved. RL more often begins from data and interaction, emphasizing sample-based evaluation, exploration, stochastic or gradient-based search, and flexible approximation of values, policies, or models. The overlap is substantial: many value-based RL algorithms are naturally viewed as sample-based approximate DP, whereas direct policy optimization may pursue the same control objective without explicitly solving the Bellman optimality equation. This book therefore uses Bellman optimality and approximate DP as its organizing viewpoint for control-oriented RL, not as a universal definition of RL.

Two meanings of model-based and model-free.

The unqualified labels *model-based* and *model-free* answer two different questions in the DP/control and modern RL literatures. The first asks how a Bellman backup, evaluation quantity, or policy improvement is computed. The second asks whether an RL algorithm uses an explicit predictive model for planning or for generating synthetic experience. These questions are related but not interchangeable, and different components of a single controller may fall on different sides. To avoid ambiguity, this book states the relevant axis explicitly—Bellman/DP implementation or modern RL architecture—whenever the distinction matters. Table 1.6 summarizes the two usages.

Bellman/DP implementation. In the usage emphasized by Bertsekas (2025), an explicit mathematical transition and cost model can be used to evaluate a Bellman expectation—exactly or numerically—or to propagate candidate controls in lookahead. A sample-based implementation instead estimates the required quantities from transitions produced by a physical system or a simulator. A Q-factor or a directly represented policy can also remove the need for explicit-model action selection. A simulator still embodies a model of the environment; *sample-based* here describes how the algorithm accesses that model, not a claim that no model exists anywhere.

Modern RL usage. In contemporary RL, *model-based RL* ordinarily means that an explicit known, learned, or continually learned predictive model is used counterfactually: the algorithm evaluates candidate future action sequences or generates synthetic rollouts. *Model-free RL* learns a value or policy from experience without using such a predictive model for planning or generating synthetic experience. It may nevertheless collect that experience from a simulator; in that case the simulator serves as the environment interface rather than as a predictive object exposed to the agent. Improved sample efficiency is a common motivation for model-based RL,

Table 1.6: Two axes behind model-based/model-free terminology.

Bellman/DP implementation. How are backups and policy improvements computed?	
<i>Explicit-model computation.</i> A transition and cost model is evaluated or propagated to compute an expectation, a minimizing action, or a future trajectory.	<i>Sample-based computation.</i> Costs and next states from a physical system or simulator are used directly as samples; the update need not evaluate a transition law.
Modern RL architecture. Is a predictive model used for planning or synthetic experience?	
<i>Model-based RL.</i> A known or learned predictive model supports current-state planning or synthetic rollouts.	<i>Model-free RL.</i> A value or policy is learned from experience without using an explicit predictive model for planning or synthetic rollouts.

not its definition (Sutton and Barto 2018).

The two classifications can assign different labels to different components of one method. For example, transitions generated from a learned model may be used in a sample-based Bellman update: the update is sample-based in the first sense, while the overall algorithm is model-based RL in the second.

Reading MPC on the two axes. Classical MPC uses known dynamics and costs to predict candidate trajectories, so its online action selection is explicit-model planning. It lies on the predictive-model side of the second question as well, but it is not thereby an RL algorithm or an online-learning method; *model-based control* or *known-model planning* is the accurate label. MPC with learned dynamics remains explicit-model planning at deployment and becomes part of a model-based RL architecture when learning and using that model are part of the RL procedure. If the model is revised during operation, online model learning is added as a separate timing property.

Known-model MPC with a critic trained from real transition samples deliberately combines the two sides: critic learning uses sample-based Bellman information, whereas online action selection uses explicit-model planning. Using the critic as a terminal cost does not make the MPC planner model-free. By contrast, selecting $u \in \arg \min_u \tilde{Q}(x, u)$ without propagating a transition model is action selection without explicit model propagation, not forward-dynamics lookahead. These qualified descriptions reveal the information and computation used by the controller instead of hiding them behind one overloaded label.

1.5 How the Book Develops the Framework

The remainder of the book follows two connected questions: where approximation or learning enters the control architecture, and how offline knowledge is used, improved, or adapted during closed-loop operation. Chapters 2–6 build the DP, approximation, value, lookahead, and policy foundations. Chapters 7–9 address uncertain models, learned-model control, stability, safety, and online adaptation. Chapters 10–11 return to multiagent mobility and integrated case studies.

Table 1.7: Questions developed across the remainder of the book.

Chapters	Central question
2–3	What do exact DP equations require, why is approximation necessary, and how should a parametric approximator be fitted for control?
4–6	How can values and policies be approximated or learned and combined with the linear quadratic regulator (LQR), MPC, rollout, and online lookahead to improve closed-loop decisions?
7–8	How can uncertain dynamics be identified or learned, and how should a learned model be used for prediction, planning, synthetic data, and control?
9	How can stability, constraints, and safety be assessed or enforced when values, policies, or models are updated online?
10–11	How do the ideas scale to interacting agents and integrated mobility applications, and what limitations remain?

Five questions provide a reading discipline across algorithms: What is the declared control problem? What object is approximated? Where do the data and model information come from? What computation occurs online? What evidence supports closed-loop performance and safety? Asking these questions makes it possible to connect classical control, approximate DP, MPC, and modern RL without erasing their differences.

Chapter 2 next develops the exact DP reference from which the later approximations and online improvements are derived.

Bibliography

- Bertsekas, D. P. (2019). *Reinforcement Learning and Optimal Control*. Belmont, MA: Athena Scientific. ISBN: 9781886529397.
- (2025). *A Course in Reinforcement Learning*. 2nd ed. Nashua, NH: Athena Scientific. ISBN: 9781886529298.
- Choi, K., J. Kim, and K.-B. Park (2025). “Generalized Model Predictive Torque Control of Synchronous Machines”. In: *IEEE/ASME Transactions on Mechatronics* 30.4, pp. 2643–2653. DOI: [10.1109/TMECH.2024.3461209](https://doi.org/10.1109/TMECH.2024.3461209).
- Choi, K., S. Shin, and M. Seo (2025). “Cooperative Merging in Mixed Traffic Based on Strategic Influence of Connected Automated Vehicles on Human-Driven Vehicle Behavior”. In: *Advanced Intelligent Systems* 7.7, 2400797. DOI: [10.1002/aisy.202400797](https://doi.org/10.1002/aisy.202400797).
- Hyundai Motor Group (May 19, 2025). *Hyundai Motor Group Navigation Integrates Traffic-Signal Information for a Safer and More Convenient Journey*. Korean-language official article. Hyundai Motor Group Journal. URL: <https://www.hyundaimotorgroup.com/ko/story/CONT0000000000176567> (visited on 08/18/2026).
- Mazzilli, V., S. De Pinto, L. Pascali, M. Contrino, F. Bottiglione, G. Mantriota, P. Gruber, and A. Sorniotti (2021). “Integrated Chassis Control: Classification, Analysis and Future Trends”. In: *Annual Reviews in Control* 51, pp. 172–205. DOI: [10.1016/j.arcontrol.2021.01.005](https://doi.org/10.1016/j.arcontrol.2021.01.005).
- Rajamani, R. (2012). *Vehicle Dynamics and Control*. 2nd ed. New York, NY: Springer. DOI: [10.1007/978-1-4614-1433-9](https://doi.org/10.1007/978-1-4614-1433-9).
- Republic of Korea (2023). *Act on Support for Mobility Innovation and Revitalization*. Act No. 19381, Article 2; Korean-language official text. Korean Law Information Center. URL: <https://www.law.go.kr/LSW/lsInfoP.do?lsiSeq=250063> (visited on 08/16/2026).
- Sciarretta, A. and A. Vahidi (2020). *Energy-Efficient Driving of Road Vehicles: Toward Cooperative, Connected, and Automated Mobility*. Cham: Springer. DOI: [10.1007/978-3-030-24127-8](https://doi.org/10.1007/978-3-030-24127-8).
- Siemens Digital Industries Software (2020). *Vehicle Electrification and Automation: Solving the E/E Dilemma of Electric and Autonomous Vehicles*. Siemens Digital Industries Software. URL: <https://resources.sw.siemens.com/en-US/white-paper-vehicle-electrification-and-automation/> (visited on 08/18/2026).
- Sutton, R. S. and A. G. Barto (2018). *Reinforcement Learning: An Introduction*. 2nd ed. Cambridge, MA: MIT Press. URL: <http://incompleteideas.net/book/the-book-2nd.html>.